

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Constraint Based Problem Solving

Code of Conduct:

*I P.Sai Ganesh Reddy(192525111) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

P.Sai Ganesh Reddy

Annexure A

Short Answer Test

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.
2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.
3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.
4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.
5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

6. You are given a large dataset of financial transactions containing duplicate records, inconsistent timestamps, and invalid entries. Design a data cleaning pipeline using NumPy and Pandas with constraints: (i) remove duplicates without losing valid data, (ii) standardize time formats, and (iii) filter out invalid transactions. Explain how indexing, grouping, and sorting ensure data integrity.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

ANSWER:

A constraint-based cleaning pipeline can be designed using **NumPy and Pandas** to ensure that the patient dataset contains no missing values, valid numerical values, and standardized categorical values. The question specifically requires these three constraints.

Steps

Step 1: Load the CSV file

```
import pandas as pd
import numpy as np
df = pd.read_csv("patient_records.csv")
```

Step 2: Check missing values

```
print(df.isnull().sum())
```

Missing numerical values can be replaced with the median:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["BloodPressure"] = df["BloodPressure"].fillna(
    df["BloodPressure"].median()
)
```

Categorical missing values can be replaced with the mode:

```
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
```

Step 3: Enforce medically valid ranges

For example:

```
df = df[(df["Age"] >= 0) & (df["Age"] <= 120)]
df = df[(df["HeartRate"] >= 30) & (df["HeartRate"] <= 220)]
df = df[(df["Temperature"] >= 35) & (df["Temperature"] <= 42)]
```

Step 4: Standardize categorical fields

```
df["Gender"] = df["Gender"].str.strip().str.lower()
df["Gender"] = df["Gender"].replace({
    "m": "male",
    "f": "female"
})
```

Step 5: Detect outliers

```
Q1 = df["Age"].quantile(0.25)
```

```
Q3 = df["Age"].quantile(0.75)
IQR = Q3 - Q1
df = df[
    (df["Age"] >= Q1 - 1.5 * IQR) &
    (df["Age"] <= Q3 + 1.5 * IQR)
]
```

Step 6: Use indexing and filtering

```
df.loc[df["Age"] < 0, "Age"] = np.nan
```

Filtering ensures that invalid records are identified and removed or corrected.

Step 7: Use aggregation for validation

```
print(df[["Age", "HeartRate", "Temperature"]].agg(
    ["min", "max", "mean"]
))
```

Step 8: Final constraint check

```
print(df.isnull().sum())
print(df.describe())
```

Conclusion

The pipeline maintains data integrity by using **indexing to select records**, **filtering to enforce valid ranges**, and **aggregation to validate numerical values**. Missing values are handled systematically and categorical values are standardized.

2. **A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.**

ANSWER:

When processing a massive NumPy dataset with limited memory, the main objective is to avoid unnecessary copies and perform operations efficiently. The question requires efficient array creation, slicing, in-place operations, indexing, aggregation, and debugging.

Efficient Array Creation

```
import numpy as np
arr = np.arange(0, 1000000, dtype=np.float32)
```

Using float32 instead of float64 reduces memory consumption when the additional precision is unnecessary.

Slicing

```
subset = arr[10000:20000]
```

NumPy slicing generally creates a **view** instead of copying the data.

In-place Operations

```
arr *= 2
```

```
arr += 10
```

These operations modify the existing array instead of creating another large array.

Random Sampling

```
sample = np.random.choice(arr, size=1000, replace=False)
```

Only the required sample is stored.

Statistical Aggregation

```
mean = np.mean(arr)
```

```
minimum = np.min(arr)
```

```
maximum = np.max(arr)
```

```
std = np.std(arr)
```

Aggregation should be performed directly on the required array or slice rather than creating unnecessary intermediate arrays.

Optimized Indexing

Boolean indexing can be used carefully:

```
indices = np.where(arr > 500)[0]
```

For very large arrays, process data in chunks rather than loading everything simultaneously.

Debugging Memory Problems

```
print(arr.nbytes)
```

```
print(arr.dtype)
```

```
print(arr.shape)
```

These help identify the memory used by an array.

For performance problems:

- Avoid unnecessary copies.
- Use slicing instead of copying.
- Use in-place operations.
- Select an appropriate data type.
- Process massive data in chunks.
- Profile slow operations.

Conclusion

Efficient array creation, slicing, appropriate data types, in-place operations, optimized indexing, and aggregation can significantly reduce memory usage while maintaining computational efficiency.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

ANSWER:

Two DataFrames can be combined using Pandas `merge()`. The major constraints are **no duplicate customer IDs, consistent indexing, and preservation of valid records.**

Example

```
import pandas as pd
transactions = pd.read_csv("transactions.csv")
demographics = pd.read_csv("demographics.csv")
```

Check Duplicate IDs

```
print(transactions["CustomerID"].duplicated().sum())
print(demographics["CustomerID"].duplicated().sum())
```

Remove duplicates when appropriate:

```
transactions = transactions.drop_duplicates(subset="CustomerID")
demographics = demographics.drop_duplicates(subset="CustomerID")
```

Combine DataFrames

```
df = pd.merge(
    transactions,
    demographics,
    on="CustomerID",
    how="outer"
)
```

An outer join preserves valid records from both DataFrames.

Validate Missing Entries

```
print(df.isnull().sum())
```

Filtering

```
missing = df[df["CustomerID"].isnull()]
```

Grouping

```
count = df.groupby("CustomerID").size()
print(count)
```

This helps identify unexpected duplicate customer IDs.

Sorting

```
df = df.sort_values("CustomerID")
```

Consistent Indexing

```
df = df.reset_index(drop=True)
```

Debugging Mismatched IDs

```
ids1 = set(transactions["CustomerID"])
```

```
ids2 = set(demographics["CustomerID"])
```

```
print("Only in transactions:", ids1 - ids2)
```

```
print("Only in demographics:", ids2 - ids1)
```

Conclusion

Using merge(), duplicate checking, grouping, filtering, sorting, and index resetting ensures that customer records are combined correctly while preserving valid data.

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

ANSWER:

The objective is to calculate statistical measures efficiently while maintaining numerical accuracy and avoiding redundant calculations.

NumPy Implementation

```
import numpy as np
```

```
data = np.array([10, 20, 30, 40, 50], dtype=np.float64)
```

```
mean = np.mean(data)
```

```
median = np.median(data)
```

```
variance = np.var(data)
```

```
print("Mean:", mean)
```

```
print("Median:", median)
```

```
print("Variance:", variance)
```

Pandas Implementation

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "Marks": [10, 20, 30, 40, 50]  
})
```

```
print("Mean:", df["Marks"].mean())
```



```
print("Median:", df["Marks"].median())  
print("Variance:", df["Marks"].var())
```

Filtering Irrelevant Data

For example, values outside the valid range can be excluded:

```
filtered = df.loc[  
    (df["Marks"] >= 0) &  
    (df["Marks"] <= 100)  
]
```

Indexing

```
values = filtered["Marks"]
```

Only the required column is selected, reducing unnecessary processing.

Aggregation

```
result = filtered["Marks"].agg(  
    ["mean", "median", "var"]  
)  
print(result)
```

Maintaining Accuracy

- Use an appropriate floating-point data type.
- Avoid repeated conversions.
- Remove invalid values before calculations.
- Avoid unnecessary intermediate arrays.
- Use NumPy/Pandas optimized aggregation functions.

Debugging Incorrect Results

```
print(df["Marks"].describe())  
print(df["Marks"].isnull().sum())  
print(df["Marks"].min())  
print(df["Marks"].max())
```

These checks help identify missing, invalid, or extreme values.

Conclusion:

Filtering and indexing ensure that only relevant observations are used, while NumPy and Pandas aggregation functions provide efficient statistical calculations.

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include:

(i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

ANSWER:

A complete pipeline consists of **loading, processing, filtering, grouping, sorting, aggregation, and visualization**. The assessment specifically requires efficient file I/O, correct grouping and sorting, user-defined filtering, meaningful plotting, and debugging of corrupted or inconsistent files.

Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Read Multiple Files

```
df1 = pd.read_csv("sales1.csv")
df2 = pd.read_csv("sales2.csv")
```

Step 3: Combine Data

```
df = pd.concat([df1, df2], ignore_index=True)
```

Step 4: Inspect Data

```
print(df.head())
print(df.info())
print(df.isnull().sum())
```

Step 5: Clean Data

```
df = df.drop_duplicates()
df = df.dropna()
```

Step 6: Filter Based on User Conditions

For example:

```
filtered = df[df["Sales"] > 10000]
```

Step 7: Group Data

```
summary = filtered.groupby("Category")["Sales"].sum()
```

Step 8: Sort Data

```
summary = summary.sort_values(ascending=False)
```

Step 9: Plot Results

```
summary.plot(kind="bar")
plt.title("Sales by Category")
```

```
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.show()
```

Debugging

For corrupted files:

try:

```
df = pd.read_csv("sales.csv")
```

except Exception as e:

```
print("Error:", e)
```

For inconsistent column names:

```
df.columns = df.columns.str.strip().str.lower()
```

For inconsistent data types:

```
df["Sales"] = pd.to_numeric(
    df["Sales"],
    errors="coerce"
)
```

Then missing or invalid values can be identified:

```
print(df.isnull().sum())
```

Conclusion

The pipeline efficiently loads multiple files, cleans and combines data, applies filtering, performs grouping and sorting, and generates meaningful visualizations. Validation and exception handling help identify corrupted files and inconsistent formats.

6. You are given a large dataset of financial transactions containing duplicate records, inconsistent timestamps, and invalid entries. Design a data cleaning pipeline using NumPy and Pandas with constraints: (i) remove duplicates without losing valid data, (ii) standardize time formats, and (iii) filter out invalid transactions. Explain how indexing, grouping, and sorting ensure data integrity.

ANSWER:

The financial dataset contains **duplicate records, inconsistent timestamps, and invalid transactions**. The required constraints are to remove duplicates safely, standardize timestamps, and filter invalid transactions.

Step 1: Load Data

```
import pandas as pd
import numpy as np
df = pd.read_csv("transactions.csv")
```

Step 2: Remove Duplicate Records

```
df = df.drop_duplicates()
If transaction ID uniquely identifies a transaction:
df = df.drop_duplicates(
    subset="TransactionID",
    keep="first"
)
```

Step 3: Standardize Timestamps

```
df["Timestamp"] = pd.to_datetime(
    df["Timestamp"],
    errors="coerce"
)
```

This converts valid timestamp formats into a standard Pandas datetime representation.

Step 4: Identify Invalid Timestamps

```
invalid_time = df[df["Timestamp"].isna()]
print(invalid_time)
```

Invalid timestamps can then be removed or corrected:

```
df = df.dropna(subset=["Timestamp"])
```

Step 5: Filter Invalid Transactions

For example, transaction amounts should not be negative:

```
df = df[df["Amount"] >= 0]
```

Additional constraints can be applied:

```
df = df[
    (df["Amount"] > 0) &
    (df["TransactionID"].notna())
]
```

Step 6: Indexing

```
df = df.set_index("TransactionID")
```

This provides efficient identification and access to transactions.

Step 7: Grouping

```
summary = df.groupby("AccountID")["Amount"].sum()
print(summary)
```

Grouping helps detect unusual transaction totals or duplicate patterns.

Step 8: Sorting

```
df = df.sort_values("Timestamp")
```

Sorting ensures transactions are arranged chronologically.

Step 9: Final Validation

```
print("Duplicate IDs:",
      df.index.duplicated().sum())
```

```
print("Missing timestamps:",
      df["Timestamp"].isna().sum())
print(df["Amount"].describe())
```

Conclusion

The pipeline uses **deduplication**, **timestamp conversion**, **filtering**, **indexing**, **grouping**, and **sorting** to maintain financial transaction integrity. Validation checks ensure that duplicates, invalid timestamps, and invalid transaction amounts are removed without unnecessarily losing valid records.